

RSA MultiPrime Attack

Ini adalah tipe challenge RSA yang cukup sering muncul di CTF menengah. Ide utamanya: **modulus RSA tidak dibuat dari 2 prima ($p \times q$), tetapi dari banyak prima.**

RSA Normal

Biasanya:

$$[\\ n = p \times q \\]$$

contoh:

$$p = 101$$

$$q = 113$$

$$n = 11413$$

MultiPrime RSA

Pada MultiPrime:

$$[\\ n = p_1 \times p_2 \times p_3 \times \cdots \times p_k \\]$$

contoh:

$$p = 11$$

$$q = 13$$

$$r = 17$$

$$n = 2431$$

Mengapa Muncul di CTF?

Karena pembuat challenge sering menggunakan banyak prima kecil:

$$n = p * q * r * s * t$$

daripada:

$$n = p * q$$

Akibatnya faktorisasi menjadi jauh lebih mudah.

Contoh Soal

Diberikan:

$$n = 2431$$

$$e = 17$$

$$c = 1906$$

dan diketahui:

n adalah hasil perkalian beberapa prime

Langkah 1 — Faktorkan n

$$\begin{aligned} &[\\ 2431 &= 11 \times 13 \times 17 \\ &] \end{aligned}$$

Jadi:

$$p = 11$$

$$q = 13$$

$$r = 17$$

Langkah 2 — Hitung $\phi(n)$

Untuk multiprime:

$$\begin{aligned} &[\\ \phi(n) &= (p-1)(q-1)(r-1) \\ &] \end{aligned}$$

Maka:

$$\begin{aligned} &[\\ \phi(n) &= 10 \times 12 \times 16 \\ &= 1920 \\ &] \end{aligned}$$

Langkah 3 — Cari d

```
[  
d=e^{-1}\pmod{\phi(n)}  
]
```

Python:

```
d = pow(e,-1,1920)
```

Langkah 4 — Dekripsi

```
[  
m=c^d \pmod n  
]
```

```
m = pow(c,d,n)
```

Bentuk Challenge CTF Sebenarnya

Sering terlihat seperti:

```
from Crypto.Util.number import *
```

```
flag = bytes_to_long(open("flag","rb").read())
```

```
primes = [getPrime(256) for _ in range(4)]
```

```
n = 1
```

```
for p in primes:
```

```
    n *= p
```

```
e = 65537
```

```
c = pow(flag,e,n)
```

```
print(n)
```

```
print(c)
```

Peserta mendapat:

```
n = 523948572398475...
```

$e = 65537$

$c = 938475983475983...$

Cara Mengenali MultiPrime

Kadang ada hint:

many factors

multiple primes

more than two

prime collection

RSA but bigger family

Challenge Nyata yang Sering Muncul

Misal:

```
primes = [  
    getPrime(64),  
    getPrime(64),  
    getPrime(64),  
    getPrime(64),  
    getPrime(64)  
]
```

```
n = product(primes)
```

Total modulus:

320 bit

Padahal RSA normal 320 bit dengan 2 prime cukup sulit.

Karena ada **5 faktor**, tools seperti:

factordb

yafu

sage

sympy.factorint()

langsung dapat memfaktorkan.

Solver Umum

```
from sympy import factorint
```

```
factors = factorint(n)
```

```
print(factors)
```

Misal hasil:

```
{  
    p1:1,  
    p2:1,  
    p3:1,  
    p4:1  
}
```

Hitung ϕ :

```
phi = 1
```

```
for p in factors:
```

```
    phi *= (p-1)
```

Cari d:

```
d = pow(e,-1,phi)
```

Dekripsi:

```
m = pow(c,d,n)
```

Variasi yang Sangat Populer di CTF

1. Many Small Primes

```
n = p*q*r*s*t*u*v
```

Faktorisasi mudah.

2. Consecutive Primes

$q = \text{next_prime}(p)$

$r = \text{next_prime}(q)$

Ada pola yang bisa dimanfaatkan.

3. Twin Prime + MultiPrime

$n = p \cdot (p+2) \cdot r$

Menggabungkan dua kelemahan sekaligus.

Soal Latihan MultiPrime

Diberikan:

$e = 17$

$n = 2431$

$c = 2038$

Diketahui:

n merupakan hasil perkalian 3 bilangan prima.

Tugas:

1. Faktorkan n .
2. Hitung $\phi(n)$.
3. Cari d .
4. Dekripsi c .
5. Flag:

$\text{flag}\{m\}$

Ringkasan

MultiPrime RSA menyerang RSA yang menggunakan:

[
 $n = p_1 p_2 p_3 \cdots p_k$
]

bukan:

$$\begin{aligned} &[\\ n &= pq \\ &] \end{aligned}$$

Langkah penyelesaian:

1. Faktorkan semua prime penyusun n .
2. Hitung

$$\begin{aligned} &[\\ \phi(n) &= \prod (p_i - 1) \\ &] \end{aligned}$$

3. Cari

$$\begin{aligned} &[\\ d &= e^{-1} \pmod{\phi(n)} \\ &] \end{aligned}$$

4. Dekripsi ciphertext.

Ini termasuk salah satu challenge RSA yang paling sering muncul setelah **Common Modulus Attack** dan **Håstad Broadcast Attack**.